Harness Evaluation Report

DeepSeek Harness (DSH) vs Pi on 3 real bug fixes from Aigerdata/scg-cash-management

Item	Value
Date	2026-11-05
Repository	Aigerdata/scg-cash-management (1,504 commits, SAP BTP Node backend + React/Vite frontend)
Harnesses	DSH `@deepseek-ai/dsh 0.2.1-rc.3` (headless profile) · Pi `0.84.3` (print mode)
Model	`claude-sonnet-4-6` for both harnesses ($3 / 1M input, $15 / 1M output)
Tasks	3 real past bugs: 1 UI change, 1 DB change, 1 backend correction
Budget	$25 limit (Rohith). Spent $7.91 in total, including tests and the rerun.
Owner	mukesh-aigerdata, with Claude Code

1. Executive summary

Harness	Final accuracy	Total time	Total cost	Tool calls
DSH	**PASS 3 / 3**	1,147 s	$3.5055	97
Pi	**PASS 3 / 3**	653 s	$1.7078	61

- **Both harnesses fixed all 3 bugs** after the UI prompt was corrected. Accuracy is a tie on this sample.
- **Pi was about 2x faster and 2x cheaper.** It used 57% of the time and 49% of the cost of DSH. Both harnesses used the same model, so the gap comes from the harness, not the model.
- **DSH reads much more context.** It read 2.7x to 3.6x more cached tokens per task. Cache reads were the largest part of the cost.
- **Failed cases are part of this report.** In the first full run, both harnesses failed the UI task. The main cause was a gap in our task prompt, not in the harnesses. Section 6 gives the full detail.
- **Both harness UI fixes were cleaner than the real human fix.** The real fix breaks 6 existing tests. Both harness fixes pass all 135 tests in the same folder.
- **The budget guard works.** Our cost total matched Pi's own cost report exactly. A forced test of the circuit breaker stopped a live run.

Recommendation: Pi is the better base for the agent-core Worker loop on cost and speed. Accuracy needs a larger task set (20 or more tasks) before the team decides.

2. Where the bugs came from

We did not write any new bugs. Each task is a real bug that an Aigerdata developer already found and fixed in the repository history. To make a task, we put the code back to the commit just before the fix. The harness gets a bug report and must find and fix the bug itself.

This is the SWE-bench method. It gives real project difficulty and an exact answer key: the tests that the real developer added with the fix.

How we chose the 3 commits

- We read the fix commits from the last 4 months of history (more than 80 candidates).
- Each task must match one of Rohith's categories: UI change, DB change or backend correction.
- The fix commit must add or change a test. That test must **fail** on the old code and **pass** on the real fix. We proved this for all 3 tasks before any harness ran.

- The fix must be small enough to finish in 20 minutes and under $4 per run.

Category	Fix commit	Start commit (before fix)	Real fix size	Hidden test
UI Change	b1716e69	f7365a2b	1 file, +36 / -15	ReimbExpenseTab.removeLastBox.test.jsx (vitest, 4 tests)
DB Change	703e4984	82056613	3 source files + 8 generated files	api-clear-advance.test.ts (node:test, 33 tests) + DB column check
Backend Correction	ca44c11b	027eeb3f	1 file, +6 / -1	sap-stale-docs.test.ts (node:test)

3. Test method

3.1 Isolation and safety

- Each task has its own `git worktree` in `eval-worktrees/`, checked out at the start commit.
- Before every run, the script resets the worktree to the start commit. The reset command first checks that the target is a registered eval worktree.
- The scripts contain no `rm` commands. All paths are literal, and arguments use `${VAR:?}` guards.
- Each harness runs in a systemd scope with a 4 GB RAM cap and a 20-minute timeout. Runs are sequential, so only one harness uses the shared 16 GB VM at a time.
- Harnesses get a clean environment: only PATH, HOME and the Anthropic key. They never see the GitHub token. The key goes in through a file descriptor, so other VM users cannot see it with `ps`.

3.2 Grading

- The harness does not see the hidden test. After the run, the grader restores the real fix's test files. Then it runs them, so a harness cannot pass by editing or deleting tests.
- **UI:** PASS when the vitest exit code is 0 (all 4 sub-tests pass).
- **Backend:** PASS when the node:test exit code is 0.
- **DB:** PASS when the target test passes, no new test fails, and the generated HANA table `ClearAdvance.hdbtable` has the new `OSDOCUMENTS` column. 3 other tests in that file already fail at both the start commit and the real fix, so the grader allows those 3 only.

3.3 Cost tracking and circuit breaker

- Cost comes from each harness's own JSON token log. Pi reports usage on `message_end`. DSH reports it on `step_end`.
- Prices per 1M tokens: input $3.00, output $15.00, cache write $3.75, cache read $0.30. Cache tokens are billed, so we count them.
- A watchdog reads the live log every 5 seconds. It stops a run at $4.00, and it stops the whole evaluation at $25.00 total.
- The evaluation also stops when a log has no usage data, so an unknown cost can never pass silently.

4. Final results

Source: `docs/EVAL_RESULTS.md`. UI rows come from run 20261105-100542 (prompt v2). DB and backend rows come from run 20261105-093306.

Task & Category	Harness	Accuracy	Execution Time (s)	Exact USD Cost
log #274 remove last Transportation/Accommodation box (b1716e69) - UI Change	DSH	**PASS**	671	$1.7178
log #274 remove last Transportation/Accommodation box (b1716e69) - UI Change	Pi	**PASS**	410	$0.9780
persist OS review documents on ClearAdvance (703e4984) - DB Change	DSH	**PASS**	345	$1.2913
persist OS review documents on ClearAdvance (703e4984) - DB Change	Pi	**PASS**	157	$0.4695
log #243 keep posted SAP docs after resubmit (ca44c11b) - Backend Correction	DSH	**PASS**	131	$0.4964
log #243 keep posted SAP docs after resubmit (ca44c11b) - Backend Correction	Pi	**PASS**	86	$0.2603

4.1 Token detail

Task	Harness	Input	Output	Cache write	Cache read	Tool calls
UI (v2)	DSH	40	14,810	126,901	3,398,697	38
UI (v2)	Pi	26	17,448	91,649	1,241,778	24
DB	DSH	35	8,254	97,313	2,674,721	36
DB	Pi	27	6,059	41,348	744,807	28
Backend	DSH	21	6,405	46,342	755,024	23
Backend	Pi	10	4,372	36,448	193,221	9

Almost all input is served from the prompt cache, so input tokens are close to zero. Cache reads are 22% to 62% of each run's cost. DSH keeps a larger context per step and runs more steps, which explains most of its extra cost.

5. Each bug and how each harness handled it

5.1 UI Change: the last Transportation / Accommodation box cannot be removed (log #274)

The bug. On the trip reimbursement expense sheet (`web/src/components/reimbursement/ReimbExpenseTab.jsx`), the remove guard counted boxes with `prev.length > 1`. So the only Transportation box had no remove button, and the only Accommodation box had a disabled one. A second problem: a saved sheet with no boxes got a new blank box every time it opened.

The real fix (b1716e69). It removes the length guard and always renders an enabled remove button with an `aria-label`. It seeds a blank box only when the sheet was never saved, meaning no expense array has items.

Hidden test (4 sub-tests): remove the only Transportation box; remove the only Accommodation box (button name `tooltip.removeLine`); a saved sheet opens with no blank boxes; a never-saved sheet opens with one of each.

How each harness handled it (prompt v2, final run)

	DSH	Pi
Result	PASS 4 / 4	PASS 4 / 4
Time / cost	671 s / $1.7178 (38 tool calls)	410 s / $0.9780 (24 tool calls)
Approach	Added a `sheetIsSaved` flag over the 5 expense arrays. Removed the length guard in both remove functions. Added `aria-label` to both buttons.	Same design: a `sheetIsSaved` flag over the 5 arrays, unconditional filter on remove, and labelled buttons.
Side effects	Updated the existing test `ReimbExpenseTab.costObjectDefaults.test.jsx` to give its fixture a saved Transportation box.	Updated the same existing test in the same way.
Wider check	All 135 tests in the reimbursement test folder pass.	All 135 tests in the reimbursement test folder pass.

Finding: the real human fix fails 6 of those 135 tests, because it breaks `costObjectDefaults` and the developer did not update that test in the same commit. Both harnesses saw the break and updated the test fixture. So on this task, both harness fixes were more complete than the real commit.

```
// Both harnesses (prompt v2) — core of the change in ReimbExpenseTab.jsx
const sheetIsSaved =
  (Array.isArray(record?.transportations) && record.transportations.length > 0) ||
  (Array.isArray(record?.perdiems) && record.perdiems.length > 0) ||
  (Array.isArray(record?.accommodations) && record.accommodations.length > 0) ||
  (Array.isArray(record?.tripExpenses) && record.tripExpenses.length > 0) ||
  (Array.isArray(record?.entertainmentExpenses) && record.entertainmentExpenses.length > 0);
...
if (existing.length === 0) return sheetIsSaved ? [] : [defaultAccom()];
const removeAccommodation = (idx) => setAccommodations((prev) => prev.filter((_, i) => i !== idx));
```

5.2 DB Change: OS review documents are lost on a clear advance

The bug. In the clear-advance approval flow, the OS reviewer uploads supporting documents and sends them with the approve action (`POST /ClearAdvanceService/approve`, field `osDocuments`). The ClearAdvance table had no column for them, and the service did not save them. After a reload, the documents were gone.

The real fix (703e4984). It adds `osDocuments: { type: json, nullable: true }` to `app/spec/domain/clear-advance.domain.yaml` and regenerates `app/generated/`, which adds the `"OSDOCUMENTS" NCLOB` column to `ClearAdvance.hdbtable`. It also stores the array when the OS step approves.

Why it is a real DB task: the data model is generated. The harness must find the domain spec, change the schema and run the AppForge code generator (`pnpm gen`). Then it must change the service.

How each harness handled it

	DSH	Pi
Result	PASS (target test passes, no new failures, column present)	PASS (target test passes, no new failures, column present)
Time / cost	345 s / $1.2913 (36 tool calls)	157 s / $0.4695 (28 tool calls)
Schema	Added the `osDocuments` json column to the domain spec.	Added the same column to the domain spec.
Codegen	Ran `pnpm gen`. The same 8 generated files changed as in the real fix.	Ran `pnpm gen`. The same 8 generated files changed as in the real fix.
Service	Builds `extra` with `osDocuments` when `step.reviewerType === 'os'` and the body has an array.	Sets `extra['osDocuments']` under the same condition.
Own test	Wrote a new test file `api-clear-advance-os.test.ts` (74 lines).	Wrote a new test file `api-clear-advance-os.test.ts` (56 lines).

Both harnesses matched the real fix closely. One difference: the shared decision handler serves both approve and reject. The real fix stores the documents only when `kind === 'approve'`. Both harness fixes check only the OS step, so they would also store documents sent with an OS reject. The hidden test does not cover a reject, so both still pass. A code reviewer would ask for the extra condition.

5.3 Backend Correction: posted SAP document numbers disappear after a resubmit (log #243)

The bug. Payment request PAY-NPM0026100009 posted its JK/KR/AB documents to S/4. Then it was sent back and resubmitted. A "stale document" filter in `app/src/s4-write/stale-docs.ts` hid every row whose last attempt was older than the latest submit. The re-validation reused the posted documents without a new attempt, so the card showed no SAP document numbers.

The real fix (ca44c11b). Only an attempt that left nothing in S/4 can be stale. Rows that are `posted` or `pending`, or that have an `sapDocNumber`, always stay visible.

How each harness handled it

	DSH	Pi
Result	PASS	PASS
Time / cost	131 s / $0.4964 (23 tool calls)	86 s / $0.2603 (9 tool calls)
Approach	Added an early `return false` in `isStaleSapDoc` for posted/pending rows and for a non-blank `sapDocNumber` (trims whitespace, like the real fix).	Added the same early return. Its document-number check is `!= null && !== ''`, so a whitespace-only number would count as present. The real fix trims it. No test covers that case.
Own test	Added a regression test to `sap-stale-docs.test.ts`.	Added a regression test to `sap-stale-docs.test.ts`.

```
// Real fix (ca44c11b)
if (['posted', 'pending'].includes(String(row['status'] ?? ''))
    || String(row['sapDocNumber'] ?? '').trim()) return false;

// DSH
const status = String(row['status'] ?? '');
if (status === 'posted' || status === 'pending') return false;
if (String(row['sapDocNumber'] ?? '').trim()) return false;

// Pi
if (row['status'] === 'posted' || row['status'] === 'pending' ||
    (row['sapDocNumber'] != null && row['sapDocNumber'] !== '')) { return false; }
```

6. Failed cases and problems found

This section records every failure and problem during the evaluation, including our own mistakes.

6.1 UI task, prompt v1: both harnesses FAILED (run 20261105-093306)

	DSH	Pi
Result	FAIL (2 of 4 sub-tests passed)	FAIL (2 of 4 sub-tests passed)
Time / cost	579 s / $1.3837 (33 tool calls)	389 s / $0.8453 (28 tool calls)
Passed	Remove the only Transportation box; never-saved sheet opens with one box of each.	Remove the only Transportation box; never-saved sheet opens with one box of each.
Failed 1	Remove the only Accommodation box: the test looks for a button named `tooltip.removeLine`. The button had no matching accessible name.	Same test failed. Pi labelled the Accommodation button `common.remove` ("Remove"), copying the Transportation label.
Failed 2	Saved sheet opens without blank boxes: DSH treated a sheet as saved only when the `transportations` or `accommodations` key existed. The test's saved sheet has only a `tripExpenses` item, so a blank box was still added.	Same test failed, for the same reason. Pi also treated a sheet as saved only when the `transportations` or `accommodations` array existed. A missing key meant "never saved", so a blank box was still added.

Root cause: our prompt, not the harnesses. Both harnesses failed the same 2 sub-tests, which pointed at the task text. Prompt v1 gave the accessible name only for the Transportation button. It also said only "a sheet that was already saved", with no rule for what "saved" means. The hidden test needs `tooltip.removeLine` for the Accommodation button, and it counts a sheet as saved when any of 5 expense arrays has items. No harness could know the button name from the bug report.

Correction (prompt v2). We added the two interface details the test checks: both button names, and the saved-sheet rule. We did not describe the code change. In the rerun, both harnesses passed 4 / 4. Prompt v1 is kept in `eval-logs/20261105-093306/ui-b1716e69.prompt-v1.md`.

Lesson for the agent-core eval set: a hidden-test task must state every interface name that the test looks up (labels, field names, routes), or the task measures guessing, not engineering.

6.2 Circuit breaker test: DSH run STOPPED on purpose (run 20261105-093233)

To prove the breaker works on a live run, we ran DSH on the backend task with a $0.05 per-run limit and a $0.10 total limit.

- Result: STOPPED after 20 s at $0.1161. The script killed the systemd scope, aborted the evaluation with exit code 2, and wrote the reason to the results file.
- No eval scope and no `dsh` process remained after the stop.
- It overshot the limit by $0.016, because DSH reports usage only at the end of each step. At a $25 limit, the overshoot is a few cents.
- This was an intended failure. It also proved that DSH reached Anthropic through our overlay config.

6.3 Problems in our own setup (found and fixed before the scored runs)

Problem	Effect	Fix
The first sandbox plan used `git reset --hard` at the research repo root.	It could wipe uncommitted research work.	Resets run only inside verified eval worktrees.
Requested model `claude-3-5-sonnet-20241022` is retired (2025-10-28).	Every API call would fail.	Used `claude-sonnet-4-6`, which has the same $3 / $15 price.

Problem	Effect	Fix
`.env` was not ignored by git, and other users could read it (mode 664).	The API key and GitHub token could be committed or read.	Added it to `.gitignore` and set mode 600. It was never committed.
A setup command used `rm` with shell variables.	The user stopped it as unsafe.	Rewrote the scripts with literal paths, `${VAR:?}` guards and no `rm`.
A copied `vendor/appforge` submodule had a broken `.git` link.	`git status` failed in the worktrees.	Moved the link file aside (no delete) and made the framework copy read-only.
DB grader searched for `osDocuments`, but the HANA file writes `"OSDOCUMENTS"`.	The real fix was graded FAIL during verification.	Made the check case-insensitive. Verification then passed for all 3 tasks.
`(( n )) && …` lines under `set -e`.	The dry run exited early with no error message.	Replaced them with `if` statements.
Passing the key as an `env` argument.	The key would show in `ps` on the shared VM.	Passed the key through a file descriptor.

7. Cost: every dollar spent

Run ID	What	Runs	Cost
20261105-092925	Step 2: cost check, Pi on the backend task (PASS)	1	$0.3506
20261105-093233	Step 2b: circuit breaker test, DSH with $0.10 limit (STOPPED)	1	$0.1161
20261105-093306	Full suite, UI prompt v1 (UI FAIL x2, DB PASS x2, backend PASS x2)	6	$4.7465
20261105-100542	UI rerun, prompt v2 (PASS x2)	2	$2.6958
	Total	**10**	**$7.9090**

- That is 32% of Rohith's $25 limit. Each run was counted against the remaining budget: the full suite ran with a $24.50 limit and the rerun with $19.50.
- Our parser's total for the Pi check run ($0.3506) matched Pi's own reported cost ($0.35065) exactly.
- Cost per solved task on the final scored runs: DSH $1.17, Pi $0.57.

8. Limits of this evaluation

- **Small sample.** 3 tasks show the speed and cost trend. They cannot show an accuracy difference. Both harnesses scored 3 / 3.
- **One run per task.** LLM agents are not deterministic. A second run can give a different time, cost or result.
- **Default settings.** Both harnesses ran with default thinking and tools. We did not tune either one.
- **No project context files.** Pi ran with `--no-context-files` so it would not load our research `CLAUDE.md`. Neither harness read the repo's own `CLAUDE.md` on purpose.
- **Process sandbox only.** The systemd scope limits RAM and time. It does not block file access outside the worktree. The agent-core design uses gVisor containers for that.
- **OpenCode was not tested.** It was in the first plan, but Rohith's scope named DSH and Pi.

9. Next steps

- Grow the set to 20 or more tasks from `scg-*` repos, with 2 runs per task. Estimated cost: $30 to $45 at today's rate. This needs a new budget approval.
- Check each new prompt against its hidden test before any paid run, using the 6.1 lesson.
- Add OpenCode as a third harness. The script needs one new entry in `harness_cmd` and `cost.mjs`.
- Feed the findings into the agent-core ADRs: Pi-style lean context for Workers, and a cost watchdog in SessionManager.

Appendix A. Files and how to rerun

Path	Purpose
run_eval.sh	Runs the harnesses, grades them, tracks cost, writes docs/EVAL_RESULTS.md
eval/lib.sh	Task table, literal paths, worktree guard, safe reset, graders
eval/setup_worktrees.sh	Creates worktrees at start commits, installs deps, --verify proves each grader
eval/cost.mjs	Sums tokens from Pi / DSH JSON logs and prices them
eval/prompts/*.md	The bug reports given to the harnesses (UI is v2)
eval/dsh-anthropic.patch.yml	Routes DSH to Anthropic through its built-in pi-ai adapter
eval-logs/<run-id>/	Raw JSON logs, stderr, grader output and diff for every run
docs/EVAL_RESULTS.md	The final results table

```
eval/setup_worktrees.sh --verify              # $0: reset worktrees, prove graders
DRY_RUN=1 ./run_eval.sh                        # $0: print the plan and empty table
HARNESSES=pi TASK_IDS=be-ca44c11b ./run_eval.sh # one cheap run (~$0.30)
ABORT_AT_USD=25.00 ./run_eval.sh               # full suite (~$5)
```